

操作系统课程设计

文件系统管理模拟器

详细设计方案

技术栈：Java + Vue 3 存储方案：索引分配 + 位图法 交互方式：CLI + Web 图形界面

目录

(在 Word 中右键点击此处 → 选择「更新域」→ 即可自动生成带页码的完整目录)

一、引言

1.1 项目背景

操作系统是计算机系统的核心系统软件，文件系统作为其重要组成部分，承担着数据组织和存储的关键职责。本课程项目通过在内存中模拟虚拟磁盘，从零构建一个简易文件系统，以加深对操作系统文件管理原理的理解。

1.2 项目目标

本项目旨在实现四个目标：理解文件存储空间的管理机制，掌握文件物理结构和目录结构的组织方式，实现一个具备格式化、目录操作、文件操作等基本能力的简易文件系统，以及通过编码实践巩固操作系统理论知识。

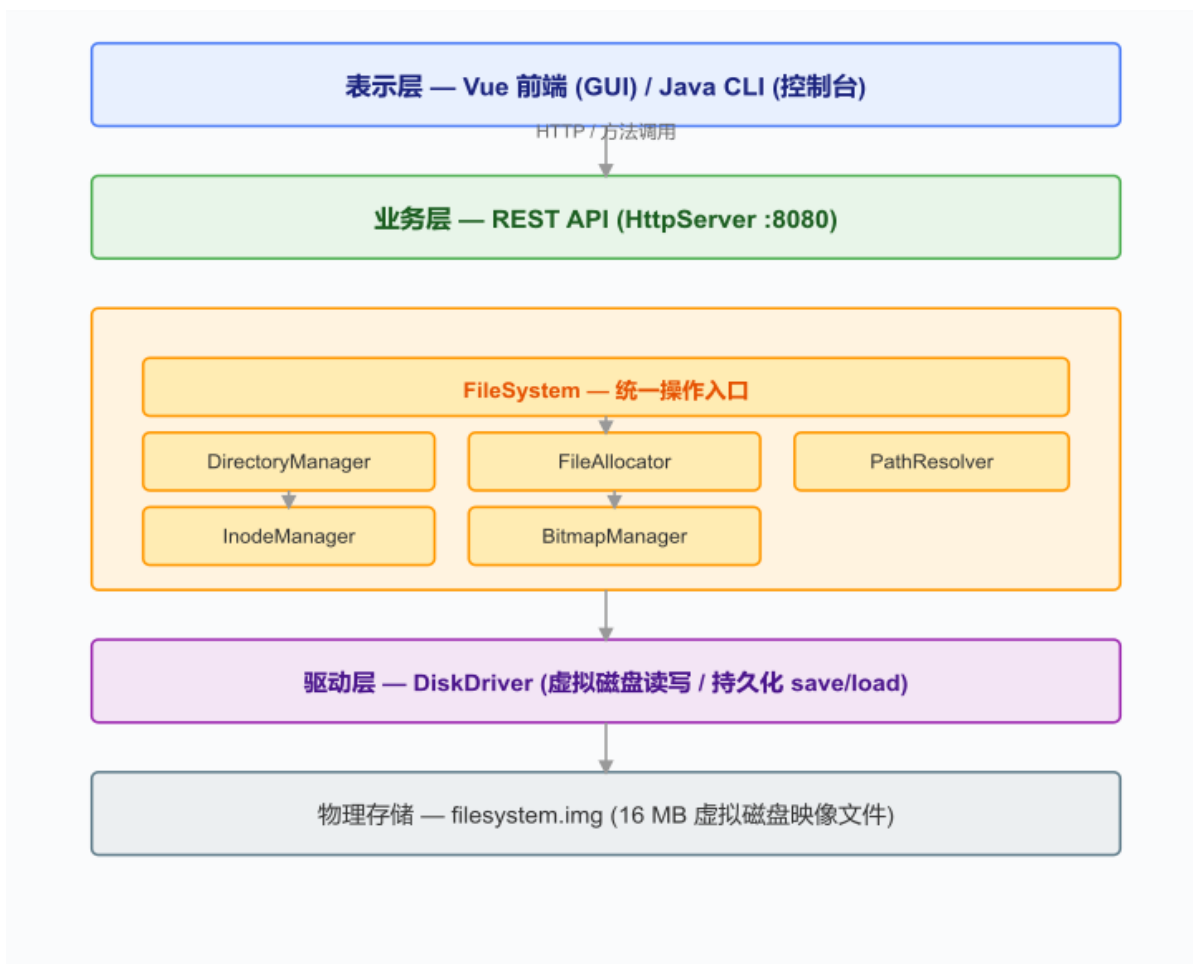
1.3 技术选型

项目采用前后端分离架构。后端使用 Java 语言开发（JDK 23），利用内置 HttpServer 提供 RESTful API；前端使用 Vue 3 框架搭配 Element Plus 组件库构建图形界面，通过 HTTP 协议与后端通信；后端同时内置命令行交互界面（CLI），可直接在控制台操作。测试使用 JUnit 5 框架。

二、系统总体架构

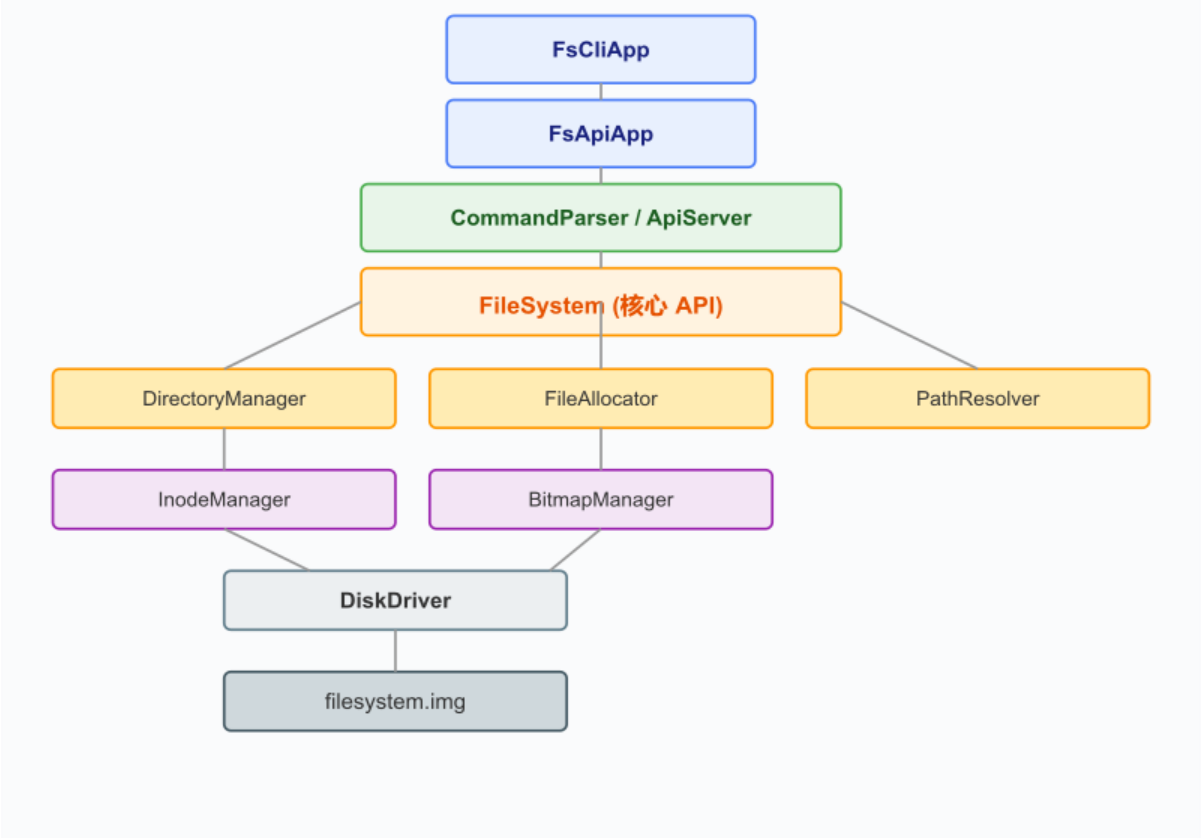
2.1 分层架构

系统整体采用分层架构，从上到下分为三层。表示层包括 Vue 前端和 Java 内置 CLI，负责与用户交互；业务层通过 REST API 接收并处理前端请求；核心层由文件系统核心类及各管理器组成，完成实际的文件系统逻辑操作。核心层内部进一步划分为文件操作接口层（FileSystem）、管理层（目录管理、文件分配、位图管理、Inode 管理等）和驱动层（DiskDriver 虚拟磁盘读写与持久化）。



2.2 模块依赖关系

CLI 和 Vue 前端均通过 FileSystem 类访问文件系统。FileSystem 依赖于目录管理器、文件分配器和路径解析器；目录管理器依赖 Inode 管理器和文件分配器来管理目录数据；文件分配器协调位图管理器和 Inode 管理器完成盘块分配与文件数据读写；所有模块最终都通过 DiskDriver 访问底层的虚拟磁盘字节数组。



2.3 包结构

包名	用途	类数
com.fs.model	数据模型（超级块/Inode/目录项等）	9
com.fs.driver	虚拟磁盘驱动 + 序列化工具	2
com.fs.manager	业务管理器（位图/Inode/目录/路径）	5
com.fs.fs	文件系统核心 API	1
com.fs.cli	CLI 命令解析器	1
com.fs.api	REST API 服务器 + JSON 工具	2

三、核心数据结构设计

3.1 虚拟磁盘布局

虚拟磁盘大小为 16 MB，以 512 字节为一个盘块，共 32,768 个盘块。第 0 块为超级块，记录文件系统的全局配置与状态信息。第 1 至 8 块为位图区，共 4,096 字节，每个比特位对应一个盘块的分配状态。第 9 至 136 块为 Inode 表区，共 128 个盘块，固定存放 1,024 个 Inode（每个 64 字节）。第 137 块起为数据区，用于存储文件和目录的实际内容。

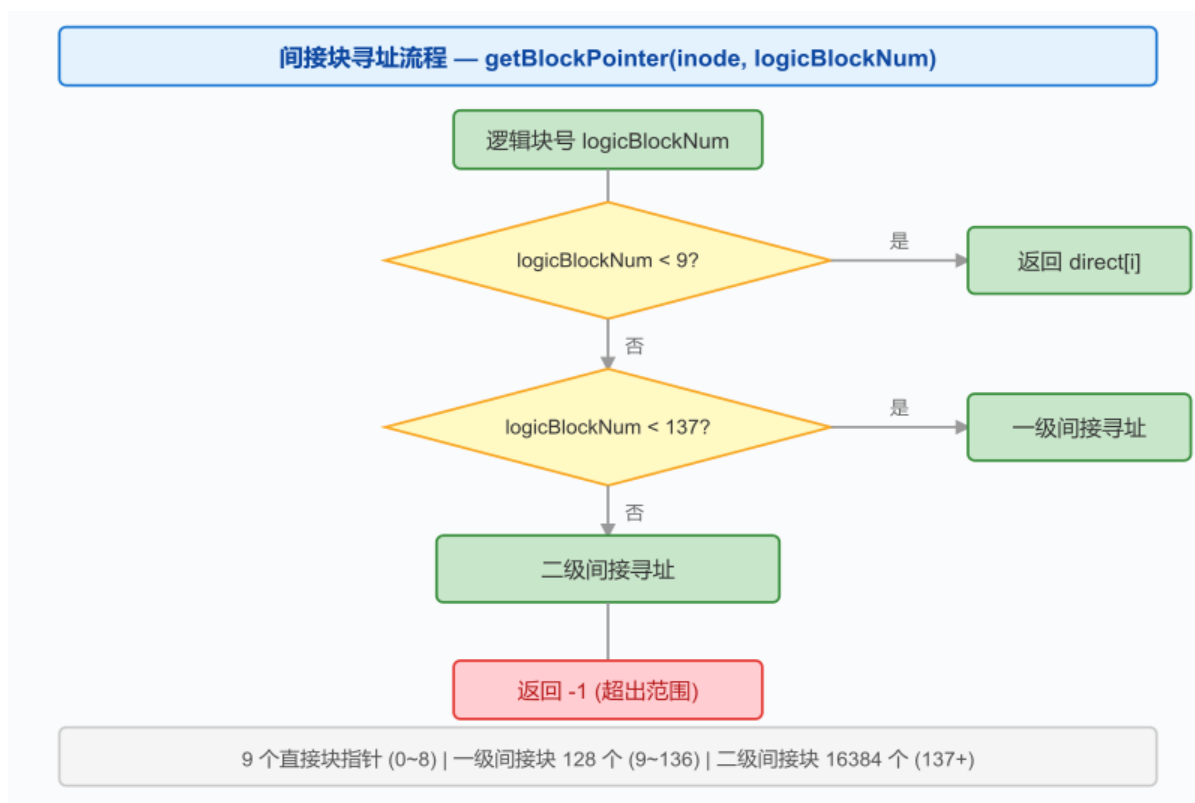
区域	块号范围	大小	说明
超级块	Block 0	512 B	魔数、总块数、位图/Inode 起始位置
位图区	Block 1 ~ 8	4,096 B	32,768 位，每位对应一个盘块
Inode 表	Block 9 ~ 136	65,536 B	1,024 个 Inode × 64 B
数据区	Block 137 ~ 32767	~16 MB	文件与目录数据

3.2 超级块

超级块位于虚拟磁盘首块，是文件系统的元数据核心，对应 SuperBlock 类。其包含魔数字段（0xF1E5F1E5）用于校验文件系统有效性，磁盘布局参数（总盘块数、盘块大小、各区域起始位置），以及动态信息（当前空闲块数和空闲 Inode 数）。

3.3 索引节点 (Inode)

每个文件或目录对应一个 Inode，记录其元数据和数据块索引，对应 Inode 类，固定 64 字节。Inode 包含文件类型（空闲/文件/目录）、文件大小、时间戳以及盘块指针数组。盘块指针采用三级索引：9 个直接块指针可直接访问前 9 块；超出时启用一级间接块指针（指向一个存储 128 个盘块号的间接块）；若文件继续增长则启用二级间接块指针（通过两级间接索引最多 16,384 个盘块）。文件最大可达约 8 MB。



3.4 位图

位图用于管理空闲盘块，共 4,096 字节（8 个盘块），其中每个比特位对应一个盘块的分配状态：0 为空闲，1 为已分配。位图在内存中维护缓存，分配或回收盘块时先修改缓存，再通过 `saveBitmap()` 写回磁盘。

3.5 目录项

目录是一种特殊文件，其数据块中存放目录项数组。每个目录项固定 48 字节，包含 Inode 编号、文件名（最长 40 字节）和文件类型。每个盘块可存放 10 个目录项。

3.6 文件描述符

文件打开时系统分配一个文件描述符，记录 Inode 编号、当前读写偏移量和打开模式（只读/只写/读写）。文件描述符表最多支持 64 个文件同时打开。

四、模块详细设计

4.1 虚拟磁盘驱动 (DiskDriver)

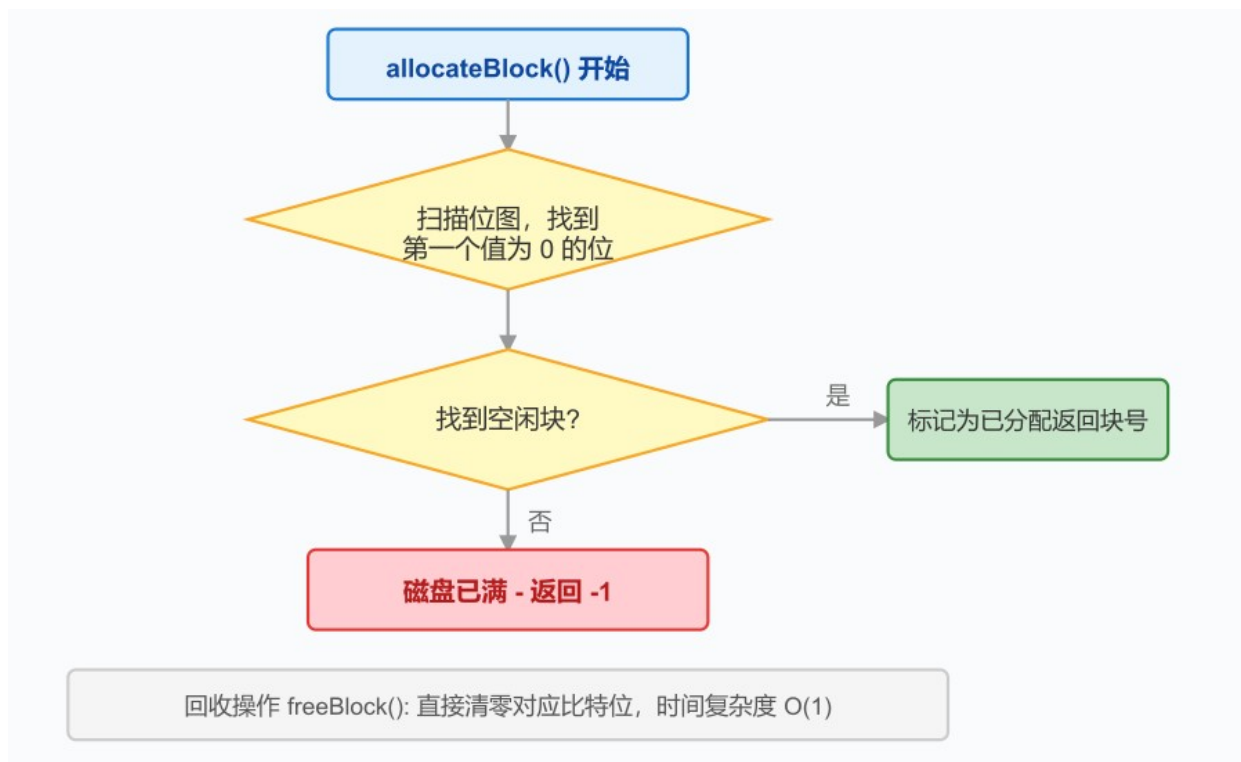
DiskDriver 是文件系统的最底层模块，内部维护一个 16 MB 字节数组作为虚拟磁盘。提供 `readBlock(int)` 和 `writeBlock(int, byte[])` 两个核心方法进行盘块级读写，以及 `readInt`、`writeInt`、`readByte`、`writeByte` 等便捷方法按偏移量读写基本数据类型。持久化由 `saveToFile()` 和 `loadFromFile()` 实现，加载时会校验超级块魔数以确保映像文件有效。

4.2 序列化工具 (Serializer)

Serializer 提供一组静态方法用于数据结构与字节数组互转。`writeSuperBlock` 将超级块各字段写入磁盘首块；`writeInode` 根据编号定位到 Inode 表中的对应位置；`dirEntryToBytes` 将目录项转换为 48 字节数组。对应的 `read` 方法完成反向操作。

4.3 位图管理器 (BitmapManager)

`allocateBlock()` 顺序扫描位图，找到第一个空闲比特位后标记为已分配并返回块号，磁盘满时返回 -1。`freeBlock(int)` 直接清零对应位，完成回收。位图操作的线程安全性通过 `synchronized` 保证。`initBitmap()` 初始化时将系统区（超级块、位图区、Inode 表区）全部标记为已分配。



4.4 Inode 管理器 (InodeManager)

`allocInode(byte)` 遍历 Inode 表找到第一个空闲槽位并初始化；`freeInode(int)` 释放指定 Inode 及其所有盘块。`getBlockPointer` 根据逻辑块号所处的范围（直接块/一级间接/二级间接）返回物理块号；`setBlockPointer` 完成反向映射。`expandFile` 在文件扩展时自动分配新盘块并建立索引。

4.5 文件分配器 (FileAllocator)

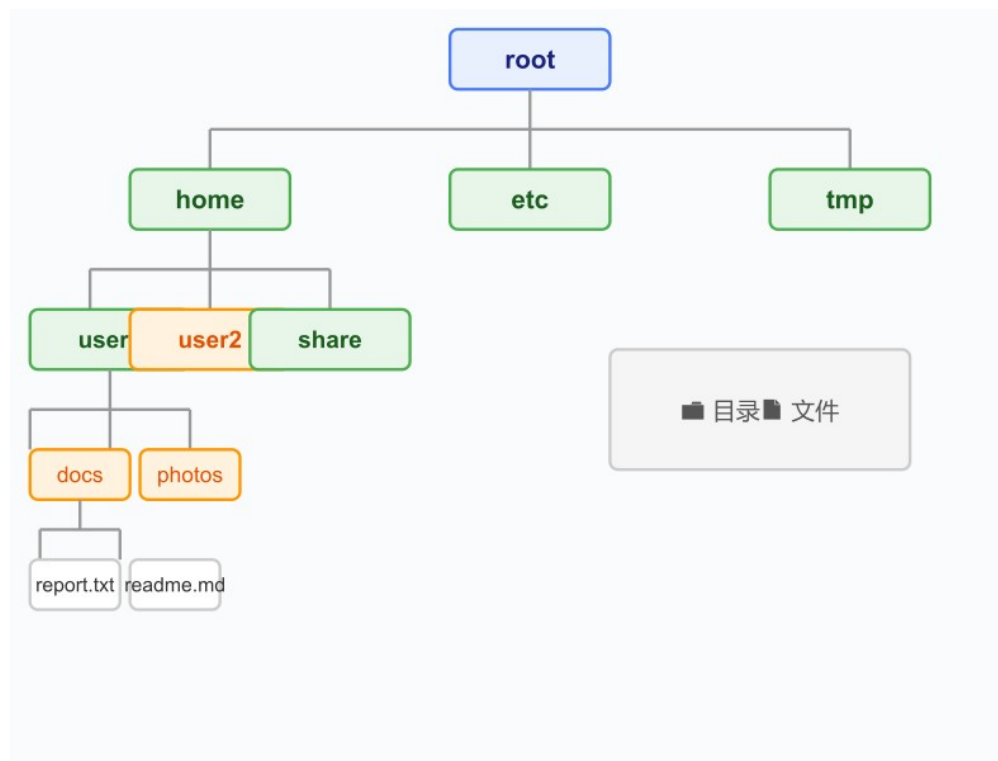
`readFileData(int, int, int)` 从指定偏移读取连续的文件数据，自动处理跨块边界。`writeFileData` 在写入时自动扩展文件并为新逻辑块分配物理盘块。`truncateFile` 支持文件截断，释放多余盘块。

4.6 路径解析器 (PathResolver)

`splitPath` 将路径按 '/' 分割，去除空分量并处理 '.' 和 '..'。`validateFileName` 使用正则验证文件名合法性。`isAbsolutePath` 判断是否为绝对路径。

4.7 目录管理器 (DirectoryManager)

目录作为特殊文件实现，其数据块中连续存放目录项。`mkdir` 分配 Inode 并在父目录中添加目录项；`rmdir` 删除空目录；`ls` 列出目录内容；`cd` 切换工作目录；`pwd` 返回当前路径字符串。`tree` 方法递归遍历目录结构生成树形文本。



4.8 文件系统核心 (FileSystem)

FileSystem 是统一操作入口，协调所有管理器完成操作。提供四类接口：系统管理（format、init、save、load、exit）、目录操作（mkdir、rmdir、ls、cd、pwd、tree）、文件操作（create、open、close、read、write、delete）和附加操作（rename、copy、move、stat）。文件描述符表在 FileSystem 中维护，长度为 64。

4.9 CLI 命令解析器 (CommandParser)

CommandParser 实现交互式命令行界面，支持 22 条命令。run 方法启动主循环，打印当前路径提示符并等待输入。executeCommand 将输入分词后匹配命令名称并分发处理。

[截图：CLI 运行界面 — 请手动插入图片]

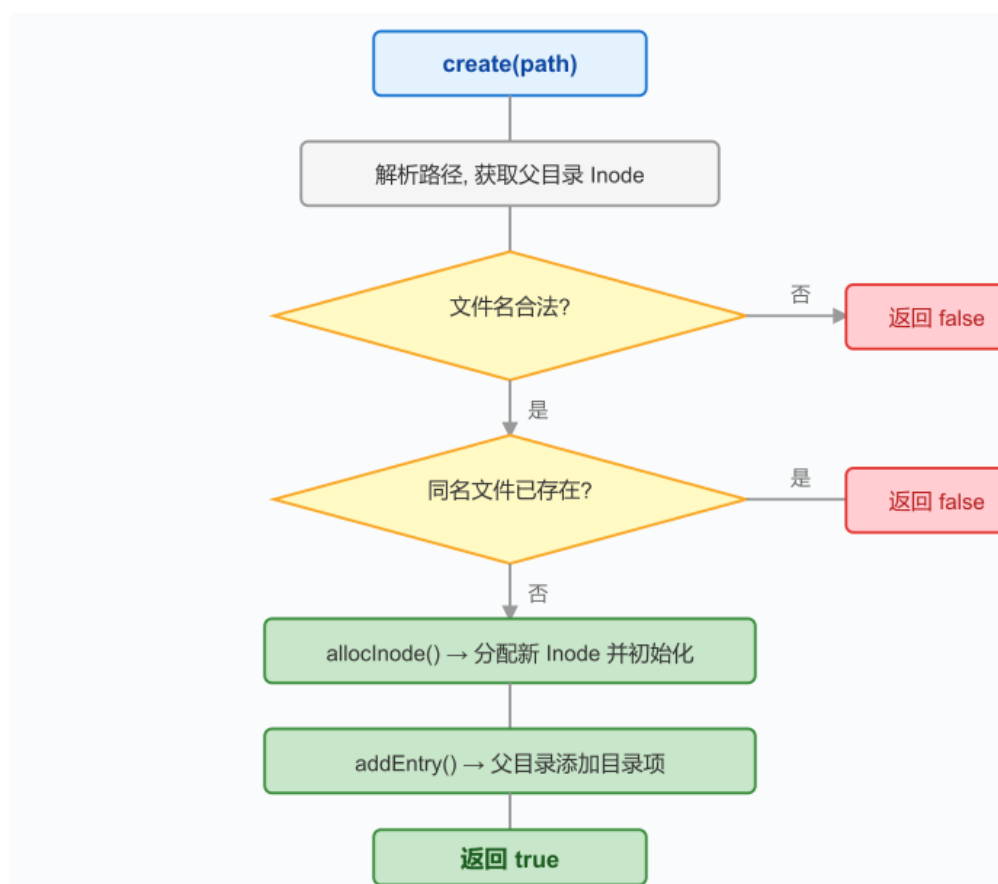
4.10 REST API 服务器 (ApiServer)

ApiServer 使用 Java 内置 HttpServer 监听 8080 端口，注册 /api 和 /api/ 两个上下文路径处理请求。dispatchBySubPath 根据 URI 子路径分发到对应处理方法。JSON 序列化使用自实现的 JsonUtil 工具类。所有端点返回统一响应格式，跨域访问在 jsonHandler 包装方法中统一处理。

五、核心算法设计

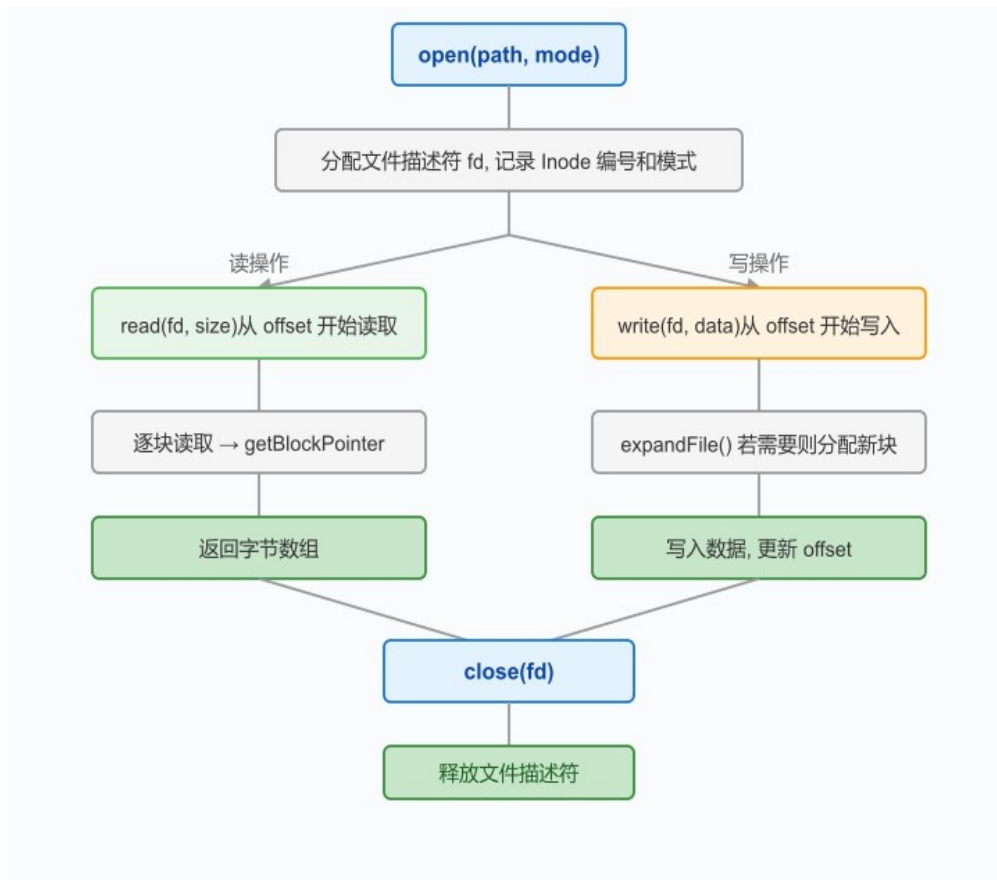
5.1 文件创建流程

create 方法首先解析路径获取父目录 Inode 和文件名，检查是否已存在同名条目。若不存在则分配新 Inode 并初始化为文件类型，最后在父目录中添加目录项。若添加目录项失败，系统自动回滚已分配的 Inode。



5.2 文件读写流程

`read(fd, size)` 检查文件描述符的打开模式，根据当前偏移量计算逻辑块范围，通过 `getBlockPointer` 获取物理块号后逐块读取数据，最后更新偏移量。`write(fd, data)` 在需要时自动调用 `expandFile` 分配新盘块以容纳写入数据。



5.3 持久化流程

`save` 方法更新超级块计数，调用 `BitmapManager.saveBitmap` 写回位图缓存，最后通过 `DiskDriver.saveToFile` 将整个字节数组写入 `filesystem.img`。系统启动时 `loadFromFile` 从磁盘文件读取数据并校验，校验通过则加载并恢复文件系统状态，否则创建一个新的空虚拟磁盘。



六、接口设计

6.1 REST API 端点

后端共提供 21 个 REST API 端点。请求和响应均使用 JSON 格式，统一结构为：{"success": true, "message": "OK", "data": ...}。

方法	路径	参数	说明
POST	/api/format	无	格式化虚拟磁盘
POST	/api/save	无	手动保存磁盘映像
GET	/api/info	无	获取系统信息
POST	/api/mkdir	{"path": ...}	创建目录
POST	/api/rmdir	{"path": ...}	删除空目录
GET	/api/ls	?path=...	列出目录内容
POST	/api/cd	{"path": ...}	切换目录
GET	/api/pwd	无	当前路径
GET	/api/tree	?path=...	树形显示
POST	/api/create	{"path": ...}	创建文件
POST	/api/open	{"path": ..., "mode": ...}	打开文件
POST	/api/close	{"fd": ...}	关闭文件
POST	/api/read	{"fd": ..., "size": ...}	读文件
POST	/api/write	{"fd": ..., "data": ...}	写文件
POST	/api/delete	{"path": ...}	删除文件
POST	/api/rename	{"oldPath": ..., "newPath": ...}	重命名
POST	/api/copy	{"srcPath": ..., "dstPath": ...}	复制文件
POST	/api/move	{"srcPath": ..., "dstPath": ...}	移动文件
GET	/api/stat	?path=...	文件信息
GET	/api/file/content	?path=...	读取文件内容
POST	/api/file/content	{"path": ..., "content": ...}	写入文件内容

6.2 CLI 命令接口

CLI 支持 22 条命令。系统管理包括 format、save、info、exit；目录操作包括 mkdir、rmdir、ls、cd、pwd、tree；文件操作包括 create、open、close、read、write、delete；附加操作包括 rename、copy、move、stat。输入 help 可查看所有命令的详细用法。

七、前端设计

7.1 页面布局

前端采用经典桌面应用布局。顶部为标题栏和系统操作按钮（保存到文件、系统信息、格式化磁盘）。左侧为目录树面板，以递归组件展示多级目录结构。右侧区域分为三部分：上方为面包屑导航和操作按钮（新建文件/目录、属性、刷新），中间为文件列表表格，下方为文件预览/编辑区域。底部为命令控制台，支持 CLI 命令输入。

[截图：Vue 前端主界面 — 请手动插入图片]

7.2 交互方式

文件列表支持左键单击进入目录或打开文件，右键弹出上下文菜单（包括重命名、复制、移动、属性、删除等操作）。目录树中点击三角形展开或折叠节点，点击名称进入目录。空白区域右键提供全局操作（新建、刷新、保存等）。所有操作按钮均带有中文文字提示。

[截图：右键菜单 — 请手动插入图片]

7.3 组件结构

前端共包含 11 个 Vue 组件。App.vue 为根组件。layout 包下包含 TopBar（标题栏与操作按钮）和 NavBar（面包屑导航与新建/属性/刷新按钮）。directory 包下包含 DirTreePanel（目录树容器）和 TreeNode（递归树节点，支持展开折叠与右键菜单）。file 包下包含 FileTable（文件列表表格，支持单击与右键）、FilePreview（文本编辑器）和 ContextMenu（Teleport 弹出式右键菜单组件）。console 包下包含 CommandConsole（命令行控制台）。

[截图：文件预览/编辑 — 请手动插入图片]

7.4 状态管理

使用 Pinia 进行状态管理，核心 Store 维护当前路径、文件列表、系统信息、预览文件状态、目录树数据和控制台日志。Store 的 Actions 封装了所有 API 调用，组件仅需调用 Store 方法即可完成操作。

八、测试方案

8.1 测试策略

测试分为单元测试和集成测试两个层次。单元测试使用 JUnit 5 对各核心类独立验证。集成测试通过完整工作流测试模拟用户操作场景，覆盖格式化、目录创建、文件读写到持久化保存与恢复的完整链路。

8.2 测试覆盖

共 7 个测试类、50 个测试用例，均已通过验证。

测试类	测试数	覆盖内容
DiskDriverTest	5	块读写、边界检查、持久化保存与加载
BitmapManagerTest	5	位图初始化、分配回收、持久化
InodeManagerTest	7	Inode 生命周期、直接/间接块指针
PathResolverTest	5	路径解析、文件名验证、绝对/相对路径
DirectoryManagerTest	10	目录创建删除、导航、条目查找、树形显示
FileSystemTest	8	文件操作全流程、扩展操作
FullWorkflowTest	10	完整工作流、跨目录复制、持久化恢复

8.3 集成测试场景

testSaveAndRestore 测试创建目录结构和文件，写入数据后保存，再重新加载验证数据完全一致。
testCopyAndVerify 测试跨目录文件复制后内容一致。testRenameAndStat 测试重命名操作前后状态变化。

[截图：JUnit 测试运行结果 — 请手动插入图片]

附录：截图索引

以下截图需在最终提交前手动补充：

- CLI 运行界面
- Vue 前端主界面
- 右键菜单
- 文件预览/编辑
- JUnit 测试运行结果